UNIVERSITÉ HASSAN II DE CASABLANCA

Faculté des Sciences Ben M'Sick

Département de Mathématiques et Informatique

Master intelligence artificielle

RAPPORT DE PROJET FINAL

Mise en Place d'une Infrastructure DevOps

avec CI/CD, Kubernetes et Monitoring

Filière : IA

Année universitaire : 2025–2026

Résumé

Ce rapport présente la réalisation complète d'un projet DevOps de bout en bout, conçu dans le cadre du Master IA à la Faculté des Sciences Ben M'Sick. L'objectif principal était de concevoir, déployer et automatiser une infrastructure cloud complète en suivant les principes et les meilleures pratiques du mouvement DevOps.

Le projet repose sur une architecture à deux machines virtuelles hébergées sur AWS EC2 (tier gratuit), orchestrées par un cluster Kubernetes léger (K3s). L'infrastructure a été provisionnée entièrement en code avec Terraform, configurée automatiquement avec Ansible, et l'application Flask a été conteneurisée avec Docker puis déployée via un pipeline CI/CD GitHub Actions automatisé.

Les principaux résultats obtenus sont les suivants : deux instances EC2 t3.micro opérationnelles formant un cluster K3s (un nœud maître et un nœud worker), une API REST Flask déployée avec deux réplicas Kubernetes, un pipeline CI/CD entièrement fonctionnel accomplissant les étapes de test, de construction d'image Docker, de publication sur Docker Hub et de déploiement automatique sur le cluster, le tout en moins de 80 secondes après chaque commit.

Ce rapport documente en détail chaque phase du projet : les choix technologiques et leur justification, la configuration de l'infrastructure, l'automatisation de la configuration des serveurs, le développement et la conteneurisation de l'application, la mise en place des manifestes Kubernetes, le pipeline CI/CD et les procédures de validation.

Table des Matières

RÉSUMÉ

1. INTRODUCTION
   1.1 Contexte et Motivation
   1.2 Objectifs du Projet
   1.3 Périmètre du Projet
   1.4 Structure du Rapport

2. ARCHITECTURE GÉNÉRALE
   2.1 Vue d'Ensemble de l'Architecture
   2.2 Choix des Technologies
      2.2.1 Fournisseur Cloud : AWS
      2.2.2 Infrastructure as Code : Terraform
      2.2.3 Configuration Management : Ansible
      2.2.4 Kubernetes : K3s
      2.2.5 CI/CD : GitHub Actions
      2.2.6 Conteneurisation : Docker
      2.2.7 Registre d'Images : Docker Hub
   2.3 Tableau Récapitulatif des Outils
   2.4 Contraintes et Limitations
      2.4.1 Contraintes Mémoire
      2.4.2 Contraintes Réseau
      2.4.3 Monitoring Non Implémenté

3. PROVISIONNEMENT DE L'INFRASTRUCTURE AVEC TERRAFORM
   3.1 Introduction à l'Infrastructure as Code
   3.2 Structure des Fichiers Terraform
   3.3 Configuration des Variables (variables.tf)
   3.4 Ressources Créées (main.tf)
      3.4.1 Provider AWS
      3.4.2 VPC (Virtual Private Cloud)
      3.4.3 Internet Gateway et Routage
      3.4.4 Sous-réseau Public
      3.4.5 Groupe de Sécurité
      3.4.6 Instances EC2
   3.5 Fichier de Sorties (outputs.tf)
   3.6 Exécution de Terraform
      3.6.1 Initialisation
      3.6.2 Planification
      3.6.3 Application
   3.7 Gestion de la Sécurité

4. CONFIGURATION AUTOMATISÉE AVEC ANSIBLE
   4.1 Introduction à Ansible
   4.2 Architecture des Playbooks

4.3 Fichier d'Inventaire (inventory.ini)
4.4 Rôle Common --- Configuration Système et Swap
4.4.1 Mise à Jour du Système
4.4.2 Installation des Paquets Prérequis
4.4.3 Configuration de la Mémoire Swap
4.5 Rôle Docker --- Installation de Docker CE
4.6 Rôle K3s --- Installation du Cluster Kubernetes
4.6.1 Installation du Nœud Maître
4.6.2 Récupération du Token de Jonction
4.6.3 Jonction du Nœud Worker
4.7 Exécution du Playbook
4.7.1 Test de Connectivité
4.7.2 Exécution Complète

5. APPLICATION FLASK REST API ET DOCKER
5.1 Présentation de l'Application
5.2 Code Source de l'Application (app.py)
5.3 Fichier des Dépendances (requirements.txt)
5.4 Tests Unitaires (tests/test_app.py)
5.5 Conteneurisation avec Docker
5.5.1 Dockerfile
5.5.2 Construction et Publication de l'Image
5.5.3 Tests Locaux de l'Image

6. ORCHESTRATION AVEC KUBERNETES (K3s)
6.1 Introduction à Kubernetes
6.2 Architecture du Cluster K3s
6.3 Manifestes Kubernetes
6.3.1 Namespace (namespace.yaml)
6.3.2 ConfigMap (configmap.yaml)
6.3.3 Deployment (deployment.yaml)
6.3.4 Service (service.yaml)
6.4 Déploiement des Manifestes
6.5 Démonstration de l'Auto-Guérison
6.6 Vérification de l'Application

7. PIPELINE CI/CD AVEC GITHUB ACTIONS
7.1 Concept de CI/CD
7.2 Architecture du Pipeline
7.3 Fichier de Configuration du Pipeline (.github/workflows/ci-cd.yml)
7.3.1 Job Test : Run tests
7.3.2 Job Build and Push : Construire et publier l'image Docker
7.3.3 Job Deploy : Déployer sur Kubernetes
7.4 Gestion des Secrets
7.5 Déclenchement Manuel
7.6 Résultat du Pipeline

8. VALIDATION ET TESTS
8.1 Stratégie de Validation
8.2 Validation de l'Infrastructure Terraform

8.3 Validation du Cluster Kubernetes
8.4 Validation de l'Application
8.5 Validation de la Mémoire Swap
8.6 Validation des Logs de l'Application

9. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS
9.1 Instance t2.micro Non Éligible au Tier Gratuit
9.1.1 Problème
9.1.2 Cause
9.1.3 Solution
9.2 Erreur de Certificat TLS K3s
9.2.1 Problème
9.2.2 Cause
9.2.3 Solution
9.3 Problème d'Image Multi-Architecture Docker
9.3.1 Problème
9.3.2 Cause
9.3.3 Solution
9.4 Clé SSH Corrompue dans les Secrets GitHub
9.4.1 Problème
9.4.2 Cause
9.4.3 Solution
9.5 Problème de Connectivité Ansible
9.5.1 Problème
9.5.2 Cause
9.5.3 Solution
9.6 Tableau Récapitulatif des Problèmes

10. CONCLUSION ET PERSPECTIVES
10.1 Bilan du Projet
10.2 Compétences Acquises
10.3 Perspectives et Améliorations Futures
10.3.1 Monitoring avec Prometheus et Grafana
10.3.2 Contrôleur Ingress
10.3.3 Charts Helm
10.3.4 Multi-Environnements
10.3.5 Gestion des Secrets avec Vault
10.3.6 Base de Données
10.4 Mot de Fin

11. ANNEXES
11.1 Annexe A --- Structure Complète du Projet
11.2 Annexe B --- Commandes de Référence Complètes
B.1 Terraform
B.2 Ansible
B.3 Kubectl
B.4 Gestion des Instances EC2
11.3 Annexe C --- Guide de Reproduction
C.1 Prérequis Locaux
C.2 Étapes de Reproduction

11.4 Annexe D --- Ressources et Références

1. Introduction

1.1 Contexte et Motivation

Le développement logiciel moderne est confronté à un défi fondamental : comment livrer des logiciels de qualité, rapidement et de manière fiable, dans un environnement de plus en plus complexe et distribué ? C'est précisément la question à laquelle le mouvement DevOps cherche à répondre.

DevOps est une culture et un ensemble de pratiques qui visent à réduire le fossé traditionnel entre les équipes de développement (Dev) et les équipes d'exploitation (Ops). En automatisant et en intégrant les processus de développement, de test, de déploiement et de surveillance des applications, DevOps permet aux organisations de livrer des applications et des services à grande vitesse, d'améliorer et de faire évoluer leurs produits plus rapidement que les organisations utilisant des processus de développement logiciel et de gestion des infrastructures traditionnels.

Dans ce contexte, ce projet final vise à mettre en pratique l'ensemble des compétences DevOps acquises au cours du Master DSBD & IA, en créant une infrastructure complète et fonctionnelle sur une plateforme cloud, en utilisant les outils les plus utilisés dans l'industrie.

1.2 Objectifs du Projet

Les objectifs principaux de ce projet sont :

- Créer et déployer une application conteneurisée à l'aide de Docker, en suivant les bonnes pratiques de sécurité et d'optimisation des images.
- Mettre en place une chaîne CI/CD entièrement automatisée permettant de détecter les problèmes tôt, de construire les images Docker et de les déployer automatiquement.
- Déployer et configurer un cluster Kubernetes (K3s) sur des machines cloud, comprenant un nœud maître (control plane) et un nœud worker.
- Utiliser Terraform pour provisionner l'infrastructure cloud de manière déclarative et reproductible.
- Utiliser Ansible pour automatiser la configuration des machines, l'installation des logiciels requis et le déploiement des composants du cluster.
- Documenter l'ensemble du processus de manière suffisamment détaillée pour permettre à quiconque de reproduire l'infrastructure.

1.3 Périmètre du Projet

Ce projet couvre les domaines suivants :

- Provisionnement de l'infrastructure cloud sur AWS (Amazon Web Services) en utilisant le tier gratuit.
- Configuration automatisée des serveurs Ubuntu 22.04 LTS.

- Installation et configuration d'un cluster Kubernetes léger (K3s).
- Développement d'une API REST simple en Python Flask.
- Conteneurisation de l'application avec Docker et publication sur Docker Hub.
- Création d'un pipeline CI/CD avec GitHub Actions.
- Déploiement de l'application sur Kubernetes avec des manifestes YAML.

Le monitoring avec Prometheus et Grafana, bien que mentionné comme optionnel dans le cahier des charges, n'a pas été implémenté en raison des contraintes de mémoire vive des instances t3.micro du tier gratuit AWS. Cette décision est documentée et justifiée dans la section dédiée aux contraintes du projet.

1.4 Structure du Rapport

Ce rapport est organisé comme suit :

1. Introduction : contexte, objectifs et périmètre du projet.
2. Architecture Générale : vue d'ensemble de l'infrastructure et choix des outils.
3. Infrastructure avec Terraform : provisionnement des ressources AWS.
4. Configuration avec Ansible : automatisation de la configuration des serveurs.
5. Application Flask et Docker : développement, tests et conteneurisation.
6. Orchestration Kubernetes : manifestes et déploiement sur le cluster.
7. Pipeline CI/CD : configuration et fonctionnement du pipeline GitHub Actions.
8. Validation et Tests : procédures de validation et résultats.
9. Difficultés Rencontrées et Solutions : problèmes rencontrés et comment ils ont été résolus.
10. Conclusion et Perspectives : bilan du projet et améliorations futures.
11. Annexes : commandes de référence, fichiers de configuration complets.

2. Architecture Générale

2.1 Vue d'Ensemble de l'Architecture

L'architecture du projet suit un modèle en couches, où chaque couche est responsable d'une partie spécifique du cycle de vie de l'application. La figure ci-dessous illustre l'architecture globale du projet.

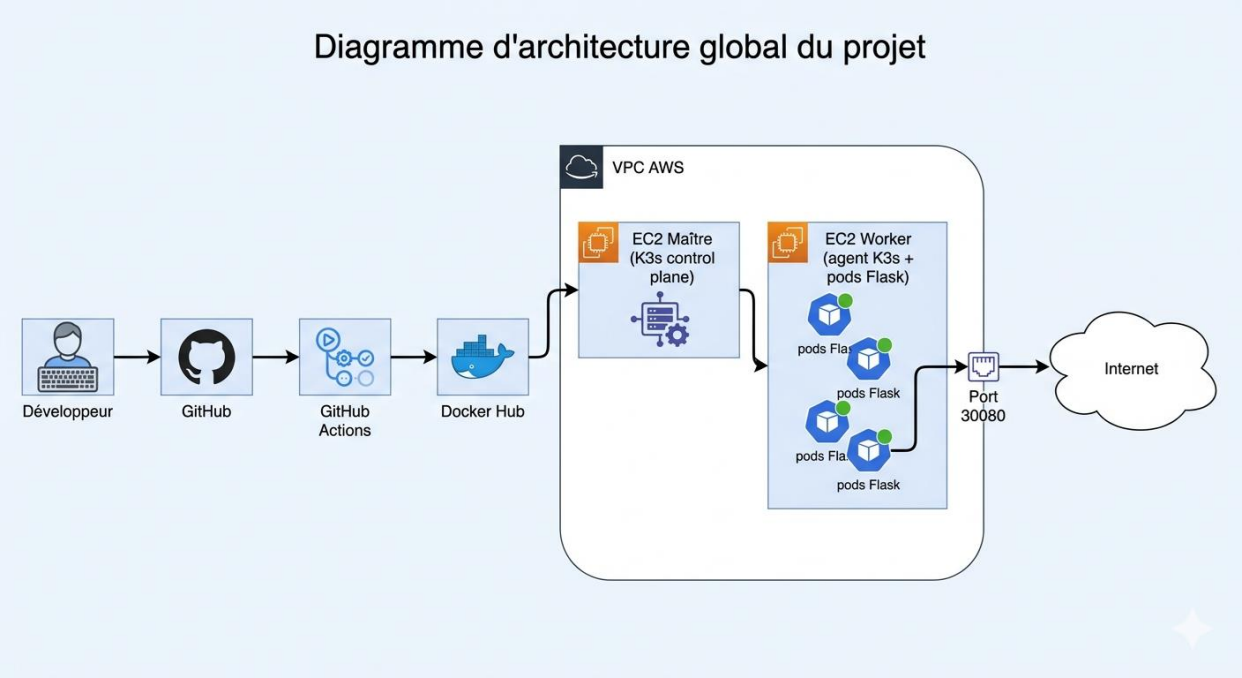


Figure 1 Diagramme d'architecture global du projet

L'architecture se décompose en quatre niveaux principaux :

- Niveau 1 — Source Control : Le code source de l'application, les fichiers Terraform, les playbooks Ansible et les manifestes Kubernetes sont tous hébergés sur GitHub dans un dépôt public.
- Niveau 2 — CI/CD : GitHub Actions orchestre le pipeline d'intégration et de déploiement continus. À chaque push sur la branche principale, le pipeline déclenche automatiquement les tests, la construction de l'image Docker, sa publication sur Docker Hub, et le déploiement sur le cluster Kubernetes.
- Niveau 3 — Registry : Docker Hub sert de registre pour les images Docker. L'image de l'application est taguée avec le hash du commit Git pour une traçabilité complète.
- Niveau 4 — Infrastructure Cloud : Deux instances EC2 sur AWS hébergent le cluster Kubernetes K3s. Le nœud maître gère le plan de contrôle (scheduling, API server, etcd) et le nœud worker exécute les pods applicatifs.

2.2 Choix des Technologies

2.2.1 Fournisseur Cloud : AWS

Amazon Web Services (AWS) a été choisi comme fournisseur cloud pour les raisons suivantes :

- Tier gratuit généreux : AWS offre 750 heures par mois d'instances t3.micro, soit suffisamment pour faire tourner 2 machines en continu pendant un mois complet.
- Écosystème mature : AWS est le fournisseur cloud leader du marché, avec une documentation exhaustive et une large communauté.
- Intégration native avec Terraform : le fournisseur AWS pour Terraform est le plus complet et le mieux documenté.
- Région us-east-1 : la région Est des États-Unis offre la latence la plus faible depuis Casablanca pour les services AWS.

2.2.2 Infrastructure as Code : Terraform

Terraform, développé par HashiCorp, est l'outil standard de l'industrie pour l'Infrastructure as Code (IaC). Il permet de décrire l'infrastructure souhaitée dans des fichiers de configuration déclaratifs (format HCL — HashiCorp Configuration Language) et de créer, modifier ou détruire les ressources correspondantes de manière cohérente et reproductible.

Les avantages de Terraform pour ce projet sont :

- Déclaratif : on décrit l'état désiré, Terraform calcule les changements nécessaires.
- Plan avant appliquer : la commande terraform plan permet de visualiser les changements avant de les appliquer.
- State management : Terraform maintient un fichier d'état qui représente l'état réel de l'infrastructure.
- Idempotence : exécuter terraform apply plusieurs fois produit le même résultat.

2.2.3 Configuration Management : Ansible

Ansible est un outil d'automatisation sans agent (agentless) qui se connecte aux serveurs via SSH pour exécuter des tâches de configuration. Contrairement à Puppet ou Chef qui nécessitent l'installation d'un agent sur chaque machine, Ansible ne requiert que Python sur les machines cibles et SSH activé.

Les avantages d'Ansible pour ce projet sont :

- Sans agent : pas d'installation supplémentaire requise sur les serveurs.
- YAML : les playbooks sont écrits en YAML, un format lisible et facile à comprendre.
- Idempotence : les tâches Ansible sont conçues pour être ré-exécutables sans effets secondaires.
- Rôles : la structure en rôles permet une organisation modulaire et réutilisable.

2.2.4 Kubernetes : K3s

Kubernetes est le système d'orchestration de conteneurs standard de l'industrie. Cependant, la distribution officielle (via kubeadm) nécessite au minimum 2 Go de RAM par nœud — une contrainte incompatible avec les instances t3.micro du tier gratuit AWS qui n'ont que 1 Go de RAM.

K3s est une distribution Kubernetes légère et certifiée CNCF (Cloud Native Computing Foundation), développée par Rancher Labs (maintenant SUSE). Elle est conçue pour fonctionner dans des environnements contraints en ressources :

- Empreinte mémoire : K3s consomme environ 480 Mo de RAM pour le nœud maître, contre plus de 2 Go pour kubeadm.
- Installation simplifiée : un seul script curl pour l'installation complète.
- Conformité : K3s est 100% conforme à la spécification Kubernetes CNCF.
- SQLite : K3s remplace etcd par SQLite pour le stockage des données du cluster, réduisant ainsi l'empreinte mémoire.

2.2.5 CI/CD : GitHub Actions

Le cahier des charges mentionnait GitLab CI, Jenkins ou GitHub Actions comme outils de CI/CD. Jenkins a été écarté car il nécessite un serveur dédié qui consommerait 500 Mo à 1 Go de RAM supplémentaires — incompatible avec les contraintes du tier gratuit AWS. GitLab CI nécessiterait un compte GitLab et une migration du dépôt.

GitHub Actions a été retenu pour les raisons suivantes :

- Zéro infrastructure : les runners sont hébergés par GitHub, sans aucune consommation de RAM sur les instances EC2.
- Gratuité : 2 000 minutes gratuites par mois pour les dépôts publics.
- Intégration native : intégration native avec Docker Hub, avec les secrets GitHub et avec le code source.
- Conformité : GitHub Actions satisfait pleinement aux exigences du cahier des charges.

2.2.6 Conteneurisation : Docker

Docker est la plateforme standard de conteneurisation. Elle permet d'encapsuler une application et toutes ses dépendances dans une image portable qui peut être exécutée de manière identique dans n'importe quel environnement — développement, test ou production.

2.2.7 Registre d'Images : Docker Hub

Docker Hub est le registre de conteneurs public de référence. La version gratuite permet de créer des dépôts publics illimités, ce qui est suffisant pour ce projet. L'image de l'application est publiée sous le nom abderrahmanelns/flask-devops-api.

2.3 Tableau Récapitulatif des Outils

Couche	Outil	Version	Justification
Cloud	AWS EC2	t3.micro	Tier gratuit — 750h/mois
IaC	Terraform	1.7.x	Standard industrie pour l'IaC
Config	Ansible	2.16.x	Agentless, YAML, rôles
Conteneur	Docker CE	28.x	Standard de conteneurisation
Kubernetes	K3s	1.34.x	Léger — 480 Mo RAM
CI/CD	GitHub Actions	N/A	Gratuit, sans serveur
Registre	Docker Hub	N/A	Public gratuit
OS	Ubuntu	22.04 LTS	LTS stable, supporté
App	Python Flask	3.0.3	Léger, simple, performant
Tests	pytest	8.x	Standard Python testing

2.4 Contraintes et Limitations

Ce projet est soumis à plusieurs contraintes importantes liées au tier gratuit d'AWS :

2.4.1 Contraintes Mémoire

Chaque instance t3.micro dispose de 1 Go de RAM. Le budget mémoire par machine est le suivant :

Composant	Consommation RAM	Machine
Système d'exploitation (Ubuntu)	~200 Mo	Maître + Worker
K3s control plane	~480 Mo	Maître
K3s agent	~150 Mo	Worker
Pods Flask (2 réplicas)	~64 Mo chacun	Worker
Mémoire swap (fichier)	2 Go (disque)	Maître + Worker

Pour pallier les risques de manque de mémoire vive (OOM — Out Of Memory), un fichier swap de 2 Go a été créé sur chaque machine via Ansible. Bien que le swap soit plus lent que la RAM, il permet d'éviter les crashes lors de pics d'utilisation mémoire.

2.4.2 Contraintes Réseau

AWS octroie 1 Go de transfert de données sortant gratuitement par mois. Pour respecter cette limite, aucun test de charge n'a été effectué et le nombre de pulls d'images Docker a été minimisé.

2.4.3 Monitoring Non Implémenté

La stack de monitoring (Prometheus + Grafana via Helm) a été évaluée mais non implémentée. Prometheus seul consomme entre 200 et 400 Mo de RAM, et Grafana entre 100 et 200 Mo supplémentaires. L'ajout de ces composants aurait provoqué des situations OOM fréquentes sur les nœuds t3.micro. Cette fonctionnalité est identifiée comme une amélioration future à implémenter sur des machines avec plus de ressources.

3. Provisionnement de l'Infrastructure avec Terraform

3.1 Introduction à l'Infrastructure as Code

L'Infrastructure as Code (IaC) est une approche qui consiste à gérer et provisionner l'infrastructure informatique à travers des fichiers de configuration lisibles par les humains et les machines, plutôt qu'à travers des processus manuels ou des interfaces graphiques. Cette approche apporte plusieurs avantages fondamentaux pour ce projet :

- Reproductibilité : l'infrastructure peut être recréée à l'identique à tout moment, sur n'importe quel compte AWS.
- Versionnement : les fichiers Terraform sont stockés dans Git, permettant de tracer l'évolution de l'infrastructure.
- Collaboration : plusieurs personnes peuvent travailler sur la même infrastructure sans conflits.
- Documentation vivante : les fichiers Terraform constituent une documentation précise et toujours à jour de l'infrastructure.

3.2 Structure des Fichiers Terraform

Le code Terraform est organisé dans le répertoire terraform/ avec la structure suivante :

```
terraform/
├── main.tf          # Ressources principales (VPC, EC2, SG, etc.)
├── variables.tf     # Déclaration des variables
└── outputs.tf       # Sorties (IPs publiques, commandes SSH)
```

3.3 Configuration des Variables (variables.tf)

Le fichier variables.tf centralise tous les paramètres configurables de l'infrastructure. Cela permet de modifier facilement la configuration sans toucher au code principal :

```
variable "aws_region" {
  description = "Région AWS"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "AMI Ubuntu 22.04 LTS pour us-east-1"
  type        = string
  default     = "ami-00de3875b03809ec5"
}
```

```
variable "instance_type" {
  description = "Type d'instance EC2 - t3.micro éligible tier gratuit"
  type        = string
  default     = "t3.micro"
}

variable "key_name" {
  description = "Nom de la paire de clés EC2"
  type        = string
  default     = "devops-project-key"
}
```

Note : *L'AMI ID (ami-00de3875b03809ec5) correspond à l'image officielle Ubuntu 22.04 LTS de Canonical pour la région us-east-1. Cet ID a été récupéré dynamiquement via la commande aws ec2 describe-images.*

3.4 Ressources Créées (main.tf)

3.4.1 Provider AWS

La configuration du provider Terraform définit la région AWS et fixe la version minimale du provider à utiliser :

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  required_version = ">= 1.7.0"
}

provider "aws" {
  region = var.aws_region
}
```

3.4.2 VPC (Virtual Private Cloud)

Le VPC est le réseau privé virtuel qui isole l'infrastructure du projet du reste d'internet et des autres clients AWS. Il utilise la plage d'adresses 10.0.0.0/16, offrant 65 536 adresses IP disponibles, largement suffisant pour ce projet :

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Name    = "devops-project-vpc"
    Project = var.project_name
  }
}
```

3.4.3 Internet Gateway et Routage

L'Internet Gateway (IGW) connecte le VPC privé à internet. La table de routage associée redirige tout le trafic sortant (0.0.0.0/0) vers l'IGW, permettant aux instances EC2 d'accéder à internet et d'être accessibles depuis l'extérieur.

3.4.4 Sous-réseau Public

Un sous-réseau public est créé dans la zone de disponibilité us-east-1a. Les instances placées dans ce sous-réseau reçoivent automatiquement une adresse IP publique grâce au paramètre map_public_ip_on_launch = true.

3.4.5 Groupe de Sécurité

Le groupe de sécurité (Security Group) agit comme un pare-feu virtuel pour les instances EC2. Il contrôle le trafic entrant et sortant au niveau de l'instance :

Type	Protocole	Port	Source	Utilisation
Entrée	TCP	22	0.0.0.0/0	SSH — accès administrateur
Entrée	TCP	6443	0.0.0.0/0	API K3s — kubectl depuis l'extérieur
Entrée	TCP	30080	0.0.0.0/0	NodePort — accès à l'application Flask
Entrée	TCP	80	0.0.0.0/0	HTTP standard
Entrée	Tout	Tout	10.0.0.0/16	Communication interne VPC
Sortie	Tout	Tout	0.0.0.0/0	Tout le trafic sortant autorisé

3.4.6 Instances EC2

Deux instances EC2 t3.micro sont créées : une pour le nœud maître K3s et une pour le nœud worker. Les deux instances utilisent la même AMI Ubuntu 22.04 LTS, le même groupe de sécurité et le même sous-réseau, avec 20 Go de stockage EBS gp2 chacune.

```
resource "aws_instance" "master" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.k3s.id]

  root_block_device {
    volume_size = 20
    volume_type = "gp2"
  }

  tags = {
    Name = "devops-project-master"
    Role = "master"
  }
}
```

3.5 Fichier de Sorties (outputs.tf)

Le fichier outputs.tf définit les valeurs que Terraform affiche après l'application. Ces sorties sont particulièrement utiles pour récupérer les adresses IP des instances créées :

```
output "master_public_ip" {
  value = aws_instance.master.public_ip
}

output "worker_public_ip" {
  value = aws_instance.worker.public_ip
}

output "ssh_master" {
  value = "ssh -i ~/.ssh/devops-project-key.pem
ubuntu@${aws_instance.master.public_ip}"
}
```

3.6 Exécution de Terraform

3.6.1 Initialisation

La commande terraform init télécharge les providers nécessaires et initialise le répertoire de travail :

```
user@DESKTOP-HMJ54M8 MINGW64 ~/devops-project/terraform (main)
$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

user@DESKTOP-HMJ54M8 MINGW64 ~/devops-project/terraform (main)
$ |
```

3.6.2 Planification

La commande terraform plan affiche un aperçu des changements qui seront appliqués, sans modifier l'infrastructure réelle. Cette étape est cruciale pour valider la configuration avant toute action :

```
Plan: 8 to add, 0 to change, 0 to destroy.


+ aws_vpc.main
+ aws_internet_gateway.main
+ aws_subnet.public
+ aws_route_table.public
+ aws_route_table_association.public
+ aws_security_group.k3s
+ aws_instance.master
+ aws_instance.worker
```

3.6.3 Application

La commande terraform apply crée effectivement toutes les ressources AWS. Après confirmation, Terraform crée les 8 ressources dans l'ordre correct (le VPC avant le sous-réseau, le sous-réseau avant les instances, etc.) :

```
$ terraform apply
aws_vpc.main: Refreshing state... [id=vpc-039cac281d5a2660e]
aws_internet_gateway.main: Refreshing state... [id=igw-093e0ef716a99e7ca]
aws_subnet.public: Refreshing state... [id=subnet-0f864414115f29063]
aws_security_group.k3s: Refreshing state... [id=sg-00d893f5c1beb42c4]
aws_route_table.public: Refreshing state... [id=rtb-08dffcf2b621d77d9]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-020a9cec388
d93f0f]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.master will be created
  + resource "aws_instance" "master" {
      + ami                                  = "ami-00de3875b03809ec5"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "devops-project-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
```

3.7 Gestion de la Sécurité

Plusieurs mesures de sécurité ont été mises en place :

- Jamais de credentials AWS dans le code : les clés d'accès AWS sont stockées dans le fichier ~/.aws/credentials via la commande aws configure, et ne sont jamais incluses dans les fichiers Terraform.
- Fichier .gitignore : le fichier .gitignore empêche de commettre accidentellement les fichiers d'état Terraform (*.tfstate) ou les fichiers de variables sensibles (*.tfvars).
- Paire de clés SSH : l'accès SSH aux instances est sécurisé par une paire de clés RSA 2048 bits. La clé privée (.pem) est stockée localement avec des permissions restrictives (chmod 400) et n'est jamais commitée dans Git.
- Utilisateur IAM dédié : au lieu d'utiliser le compte root AWS, un utilisateur IAM dédié (terraform-user) avec les permissions AdministratorAccess a été créé pour Terraform.

4. Configuration Automatisée avec Ansible

4.1 Introduction à Ansible

Ansible est un outil d'automatisation open source développé par Red Hat. Il permet de gérer la configuration des serveurs, de déployer des applications et d'orchestrer des workflows complexes, le tout sans nécessiter l'installation d'un agent sur les machines cibles. La communication se fait entièrement via SSH, ce qui simplifie considérablement la gestion de l'infrastructure.

Dans ce projet, Ansible est utilisé pour transformer les instances EC2 vierges créées par Terraform en nœuds opérationnels du cluster K3s. Il prend en charge l'installation de Docker, la configuration de la mémoire swap, l'installation et la configuration de K3s, et la récupération du fichier kubeconfig.

4.2 Architecture des Playbooks

Les playbooks Ansible sont organisés selon la structure officielle des rôles Ansible (Ansible Galaxy), ce qui favorise la modularité et la réutilisabilité :

```
ansible/
├── inventory.ini              # Inventaire des machines cibles
├── playbook.yml               # Playbook principal
└── roles/
    ├── common/
    |   └── tasks/
    |       └── main.yml       # Mise à jour système + swap
    ├── docker/
    |   └── tasks/
    |       └── main.yml       # Installation Docker CE
    └── k3s/
        └── tasks/
            └── main.yml       # Installation K3s
```

4.3 Fichier d'Inventaire (inventory.ini)

Le fichier d'inventaire définit les machines cibles et leurs paramètres de connexion. Les machines sont regroupées en deux groupes : master et worker :

```
[master]
35.172.240.19 ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/devops-project-key.pem
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

```
[worker]
3.236.90.164 ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/devops-project-key.pem
ansible_ssh_common_args='-o StrictHostKeyChecking=no'


[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

Note : *Les adresses IP présentées ici correspondent aux adresses IP publiques des instances EC2 lors des tests. Ces adresses peuvent changer si les instances sont arrêtées et redémarrées. Il convient de mettre à jour l'inventaire après chaque redémarrage.*

4.4 Rôle Common — Configuration Système et Swap

4.4.1 Mise à Jour du Système

La première tâche du rôle common met à jour le cache APT et met à niveau tous les paquets installés. Cette étape est essentielle pour s'assurer que le système dispose des dernières mises à jour de sécurité :

```
- name: Mettre à jour le cache APT
  apt:
    update_cache: yes
    cache_valid_time: 3600
  become: yes

- name: Mettre à niveau tous les paquets
  apt:
    upgrade: dist
    autoremove: yes
  become: yes
```

4.4.2 Installation des Paquets Prérequis

Plusieurs paquets sont nécessaires pour la suite de l'installation : curl et wget pour télécharger des fichiers, git pour la gestion du code source, apt-transport-https, ca-certificates et gnupg pour l'ajout de dépôts sécurisés, et python3 avec pip pour l'exécution des modules Ansible qui en ont besoin.

4.4.3 Configuration de la Mémoire Swap

La configuration de la mémoire swap est une étape critique pour ce projet. Les instances t3.micro n'ont que 1 Go de RAM, ce qui est insuffisant pour faire tourner K3s et les applications simultanément sans risque de crash OOM (Out Of Memory).

La swap est configurée selon les étapes suivantes :

12. Vérification de l'existence préalable d'un fichier swap pour assurer l'idempotence.
13. Création d'un fichier de 2 Go avec la commande fallocate.
14. Attribution des permissions restrictives (0600) au fichier swap.
15. Formatage du fichier swap avec mkswap.
16. Activation du swap avec swapon.
17. Ajout d'une entrée dans /etc/fstab pour la persistance au redémarrage.
18. Configuration du paramètre vm.swappiness à 10 pour minimiser l'utilisation de la swap en conditions normales.

```
- name: Créer un fichier swap de 2 Go
  command: fallocate -l 2G /swapfile
  become: yes
  when: not swap_file.stat.exists

- name: Définir les permissions du fichier swap
  file:
    path: /swapfile
    mode: '0600'
  become: yes

- name: Activer le swap
  command: swapon /swapfile
  become: yes
  when: not swap_file.stat.exists

- name: Rendre le swap permanent
  lineinfile:
    path: /etc/fstab
    line: '/swapfile none swap sw 0 0'
  become: yes
```

4.5 Rôle Docker — Installation de Docker CE

Docker CE (Community Edition) est installé à partir du dépôt officiel Docker pour garantir d'avoir la dernière version stable. L'installation depuis les dépôts Ubuntu par défaut donnerait une version plus ancienne (docker.io) qui n'est pas recommandée.

Les étapes d'installation de Docker sont :

19. Suppression des éventuelles anciennes versions de Docker.
20. Ajout de la clé GPG officielle Docker pour vérifier l'authenticité des paquets.
21. Ajout du dépôt Docker officiel à la liste des sources APT.
22. Installation des paquets docker-ce, docker-ce-cli et containerd.io.
23. Démarrage et activation du service Docker pour qu'il démarre automatiquement au boot.
24. Ajout de l'utilisateur ubuntu au groupe docker pour permettre l'exécution de commandes Docker sans sudo.

```
- name: Ajouter la clé GPG Docker
  shell: |
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
  args:
    creates: /usr/share/keyrings/docker-archive-keyring.gpg
  become: yes

- name: Installer Docker CE
  apt:
    name:
      - docker-ce
      - docker-ce-cli
      - containerd.io
    state: present
  become: yes
```

4.6 Rôle K3s — Installation du Cluster Kubernetes

4.6.1 Installation du Nœud Maître

K3s est installé sur le nœud maître via le script officiel d'installation. Plusieurs options importantes sont passées lors de l'installation :

- --disable traefik : désactive le contrôleur Ingress Traefik intégré à K3s pour économiser des ressources mémoire.
- --write-kubeconfig-mode 644 : rend le fichier kubeconfig lisible par l'utilisateur non-root, facilitant l'utilisation de kubectl.
- --tls-san 35.172.240.19 : inclut l'adresse IP publique du maître dans le certificat TLS du serveur API K3s, permettant la connexion depuis l'extérieur.

```
- name: Installer K3s sur le nœud maître
  shell: |
    curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server \
```

```
      --disable traefik \
      --write-kubeconfig-mode 644 \
      --tls-san 35.172.240.19" sh -
  become: yes
  when: inventory_hostname in groups['master']
```

4.6.2 Récupération du Token de Jonction

Après l'installation de K3s sur le maître, un token de jonction est généré automatiquement dans le fichier /var/lib/rancher/k3s/server/node-token. Ce token est utilisé par les nœuds workers pour s'authentifier auprès du maître et rejoindre le cluster :

```
- name: Récupérer le token K3s depuis le maître
  slurp:
    src: /var/lib/rancher/k3s/server/node-token
  register: k3s_token
  become: yes
  when: inventory_hostname in groups['master']

- name: Stocker le token comme fact Ansible
  set_fact:
    k3s_node_token: "{{ k3s_token.content | b64decode | trim }}"
  when: inventory_hostname in groups['master']
```

4.6.3 Jonction du Nœud Worker

L'agent K3s est installé sur le nœud worker en fournissant l'URL du serveur API (via l'IP privée du maître pour une communication interne au VPC) et le token de jonction récupéré à l'étape précédente :

```
- name: Installer l'agent K3s sur le worker
  shell: |
    curl -sfL https://get.k3s.io | \
      K3S_URL=https://10.0.1.178:6443 \
      K3S_TOKEN="{{ hostvars[groups['master'][0]]['k3s_node_token'] }}"
sh -
  become: yes
  when: inventory_hostname in groups['worker']
```

Note : *L'IP privée 10.0.1.178 est utilisée pour la communication interne entre le maître et le worker, car la communication intra-VPC ne génère pas de coûts de transfert de données et est plus performante que la communication via les IPs publiques.*

4.7 Exécution du Playbook

4.7.1 Test de Connectivité

Avant d'exécuter le playbook complet, la commande ansible ping permet de vérifier que les deux machines sont accessibles :

```
ansible_python_interpreter=/usr/bin/python3
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project$ cd /mnt/c/Users/user/devops-project/ansible
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ ansible all -i inventory.ini -m ping
3.236.90.164 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
35.172.240.19 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

4.7.2 Exécution Complète

Le playbook est exécuté avec l'option -v (verbose) pour afficher le détail de chaque tâche :

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ ansible-playbook -i inventory.ini playbook.yml

PLAY [Configure all nodes (common setup + swap)] ******************************************************************
TASK [Gathering Facts] *********************************************************************************ok: [35.172.240.19]
ok: [3.236.90.164]

PLAY [Configure K3s cluster] ***************************************************************************************
TASK [Gathering Facts] *********************************************************************************ok: [35.172.240.19]

PLAY [Configure K3s cluster] ***************************************************************************************
TASK [Gathering Facts] *********************************************************************************ok: [3.236.90.164]

PLAY RECAP *********************************************************************************************3.236.90.164               : ok=2    changed=0    unreachable=0    failed=0    skipped=0
 rescued=0    ignored=0
35.172.240.19              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Le résultat du PLAY RECAP confirme que toutes les tâches ont été exécutées avec succès : aucune erreur (failed=0) et aucune machine inaccessible (unreachable=0). Les 15 changements (changed=15) correspondent aux installations et configurations effectuées sur chaque machine.

5. Application Flask REST API et Docker

5.1 Présentation de l'Application

L'application développée est une API REST simple écrite en Python avec le framework Flask. Bien que simple dans sa conception, elle est représentative d'une application de microservice réelle et permet de démontrer l'ensemble du pipeline DevOps.

L'application expose trois endpoints HTTP :

Endpoint	Méthode HTTP	Description	Exemple de réponse
GET /	GET	Informations générales de l'application	{"status": "running", "version": "1.0.0"}
GET /health	GET	Vérification de l'état de santé (liveness)	{"status": "healthy", "hostname": "...", "timestamp": "..."}
GET /info	GET	Informations sur l'environnement d'exécution	{"app": "...", "pod_name": "...", "node_name": "..."}

L'endpoint /health est particulièrement important car il est utilisé par Kubernetes pour les sondes de vérification (liveness probe et readiness probe), qui permettent au cluster de détecter et de redémarrer automatiquement les pods défaillants.

L'endpoint /info expose des variables d'environnement injectées par Kubernetes au démarrage des pods (POD_NAME et NODE_NAME), permettant de vérifier sur quel pod et sur quel nœud tourne chaque requête.

5.2 Code Source de l'Application (app.py)

```
from flask import Flask, jsonify
import os
import socket
from datetime import datetime

app = Flask(  name  )

@app.route('/')
def home():
    return jsonify({
        "message": "DevOps Project API",
```

```
        "status": "running",
        "version": "1.0.0"
    })

@app.route('/health')
def health():
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.utcnow().isoformat(),
        "hostname": socket.gethostname()
    })

@app.route('/info')
def info():
    return jsonify({
        "app": "flask-devops-api",
        "environment": os.getenv("APP_ENV", "production"),
        "pod_name": os.getenv("POD_NAME", "unknown"),
        "node_name": os.getenv("NODE_NAME", "unknown")
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
```

5.3 Fichier des Dépendances (requirements.txt)

Le fichier requirements.txt fixe les versions exactes des dépendances Python pour garantir la reproductibilité des builds :

```
flask==3.0.3
gunicorn==22.0.0
```

Flask est le framework web léger qui expose les endpoints HTTP. Gunicorn (Green Unicorn) est un serveur HTTP WSGI de production pour les applications Python. Il remplace le serveur de développement intégré à Flask, qui n'est pas adapté à une utilisation en production.

5.4 Tests Unitaires (tests/test_app.py)

Des tests unitaires ont été écrits avec pytest pour vérifier le bon fonctionnement de chaque endpoint. Ces tests sont intégrés dans le pipeline CI/CD et sont exécutés automatiquement à chaque commit :

```python
import pytest
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from app import app

@pytest.fixture
def client():
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

def test_home(client):
    response = client.get('/')
    assert response.status_code == 200
    data = response.get_json()
    assert data['status'] == 'running'
    assert data['version'] == '1.0.0'

def test_health(client):
    response = client.get('/health')
    assert response.status_code == 200
    data = response.get_json()
    assert data['status'] == 'healthy'
    assert 'timestamp' in data
    assert 'hostname' in data

def test_info(client):
    response = client.get('/info')
    assert response.status_code == 200
    data = response.get_json()
    assert data['app'] == 'flask-devops-api'
```

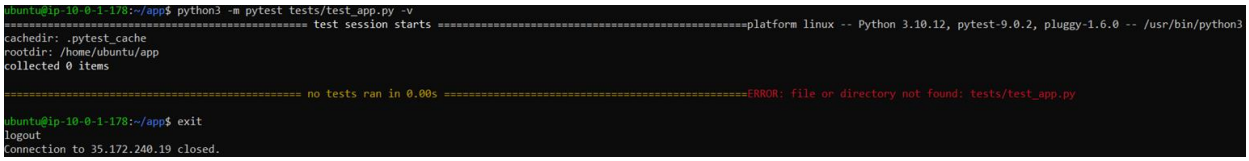


Figure 2 Tests Unitaires (tests/test_app.py)

5.5 Conteneurisation avec Docker

5.5.1 Dockerfile

Le Dockerfile décrit les étapes pour construire l'image Docker de l'application. Il suit les bonnes pratiques de construction d'images Docker :

```
# Image de base Python légère
FROM python:3.11-slim

# Répertoire de travail dans le conteneur
WORKDIR /app

# Variables d'environnement Python
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_ENV=production

# Copie et installation des dépendances en premier
# (couche cachée si requirements.txt ne change pas)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copie du code source
COPY app.py .

# Création d'un utilisateur non-root pour la sécurité
RUN adduser --disabled-password --gecos '' appuser && \
    chown -R appuser:appuser /app
USER appuser

# Port exposé par le conteneur
EXPOSE 5000

# Vérification de santé intégrée
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3
\
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:5000/health')"
```

```
# Démarrage avec Gunicorn (2 workers pour la production)
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--
timeout", "60", "app:app"]
```

Les bonnes pratiques appliquées dans ce Dockerfile sont :

- Image de base slim : python:3.11-slim réduit la taille de l'image de ~900 Mo (image complète) à ~180 Mo.
- Copie de requirements.txt en premier : Docker met en cache les couches. Si requirements.txt ne change pas, la couche d'installation des dépendances est réutilisée, accélérant les builds.
- Utilisateur non-root : l'application tourne sous l'utilisateur appuser plutôt que root, réduisant la surface d'attaque en cas de compromission.
- HEALTHCHECK : Docker surveille l'état de santé du conteneur et le marque comme unhealthy si le healthcheck échoue.
- PYTHONDONTWRITEBYTECODE : évite la création de fichiers .pyc inutiles.
- PYTHONUNBUFFERED : force Python à écrire directement dans stdout/stderr sans buffering, important pour les logs Docker.

5.5.2 Construction et Publication de l'Image

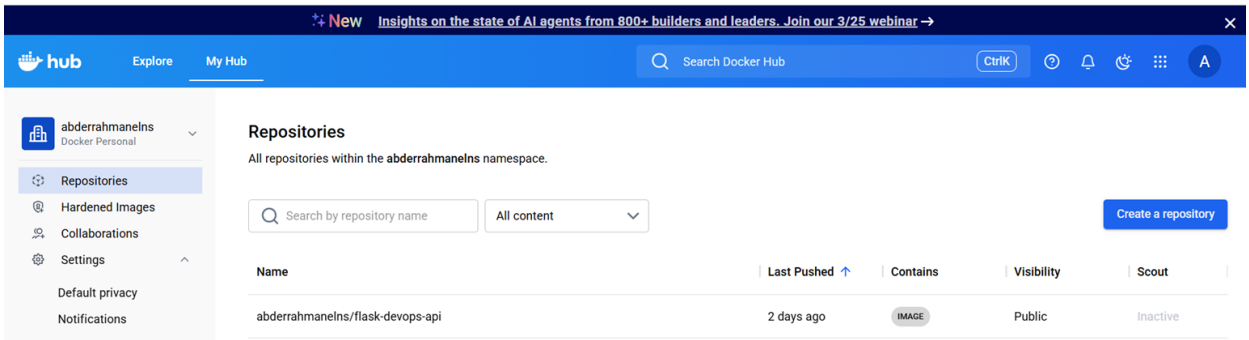
L'image Docker a été construite directement sur l'instance EC2 maître (car Docker Desktop n'était pas installé sur la machine Windows de développement) :

```
# Construction de l'image pour l'architecture linux/amd64
docker build --platform linux/amd64 -t abderrahmanelns/flask-devops-
api:latest .


# Connexion à Docker Hub
docker login -u abderrahmanelns


# Publication de l'image
docker push abderrahmanelns/flask-devops-api:latest
```

Note : *L'option --platform linux/amd64 est cruciale car sans elle, Docker peut créer une image multi-architectures (manifest list) qui n'est pas compatible avec le runtime containerd utilisé par K3s. Cette option force la création d'une image pour une seule architecture.*



5.5.3 Tests Locaux de l'Image

Avant de déployer sur Kubernetes, l'image a été testée localement sur le serveur EC2 :

```
# Démarrage du conteneur en mode détaché
docker run -d -p 5000:5000 --name flask-test abderrahmanelns/flask-
devops-api:latest

# Test des endpoints
curl http://localhost:5000/
curl http://localhost:5000/health
curl http://localhost:5000/info

# Nettoyage
docker stop flask-test && docker rm flask-test
```

```
flask-api-86794dc585-8b6tv   1/1     Running   0          19s
^Cuser@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/
{"message":"DevOps Project API","status":"running","version":"1.0.0"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/
{"message":"DevOps Project API","status":"running","version":"1.0.0"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/health
{"hostname":"flask-api-86794dc585-8b6tv","status":"healthy","timestamp":"2026-03-21T03:44:58.314693"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/info
{"app":"flask-devops-api","environment":"production","node_name":"ip-10-0-1-124","pod_name":"flask-api-86794dc585-k4hg6"}
```

6. Orchestration avec Kubernetes (K3s)

6.1 Introduction à Kubernetes

Kubernetes (K8s) est une plateforme open source d'orchestration de conteneurs, initialement développée par Google et maintenant maintenue par la Cloud Native Computing Foundation (CNCF). Elle automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées.

Les concepts fondamentaux de Kubernetes utilisés dans ce projet sont :

- Pod : la plus petite unité déployable dans Kubernetes. Un pod encapsule un ou plusieurs conteneurs Docker, avec des ressources partagées (réseau, stockage).
- Deployment : un contrôleur qui gère un ensemble de pods identiques (réplicas). Il assure que le nombre désiré de réplicas est toujours en cours d'exécution, redémarrant automatiquement les pods défaillants.
- Service : une abstraction qui expose une application s'exécutant dans des pods. Il fournit un point d'accès stable (IP et nom DNS) indépendamment des pods individuels qui peuvent changer.
- Namespace : un espace de noms virtuel qui permet de diviser un cluster Kubernetes en plusieurs environnements isolés.
- ConfigMap : un objet qui stocke des données de configuration non sensibles sous forme de paires clé-valeur.

6.2 Architecture du Cluster K3s

Le cluster K3s est composé de deux nœuds :

Nœud	Rôle	IP Privée	IP Publique	Composants K3s
ip-10-0-1-178	control-plane	10.0.1.178	35.172.240.19	API Server, Scheduler, Controller Manager, etcd, kube-proxy, containerd
ip-10-0-1-124	worker	10.0.1.124	3.236.90.164	Agent K3s, kube-proxy, containerd, pods applicatifs

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get nodes -o wide
NAME            STATUS   ROLES           AGE   VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
ip-10-0-1-124   Ready    <none>          18h   v1.34.5+k3s1   10.0.1.124    <none>        Ubuntu 22.04.5 LTS   6.8.0-1050-aws   containerd://2.1.5-k3s1
ip-10-0-1-178   Ready    control-plane   19h   v1.34.5+k3s1   10.0.1.178    <none>        Ubuntu 22.04.5 LTS   6.8.0-1050-aws   containerd://2.1.5-k3s1
```

6.3 Manifestes Kubernetes

Les manifestes Kubernetes sont des fichiers YAML qui décrivent l'état désiré des ressources du cluster. Tous les manifestes sont stockés dans le répertoire k8s/ du projet.

6.3.1 Namespace (namespace.yaml)

Un namespace dédié devops-project est créé pour isoler les ressources de l'application des autres ressources du cluster :

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: devops-project
  labels:
    project: devops-project
```

L'utilisation d'un namespace dédié permet de regrouper toutes les ressources liées à l'application, de les supprimer en une seule commande (kubectl delete namespace devops-project), et de potentiellement définir des politiques de ressources (Resource Quotas) spécifiques.

6.3.2 ConfigMap (configmap.yaml)

Le ConfigMap stocke les variables de configuration non sensibles de l'application. En séparant la configuration du code, il est possible de modifier le comportement de l'application sans reconstruire l'image Docker :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: flask-config
  namespace: devops-project
data:
  APP_ENV: "production"
```

6.3.3 Deployment (deployment.yaml)

Le Deployment est le manifeste le plus important car il décrit comment l'application doit être déployée et maintenue. Il configure notamment les réplicas, les ressources, les sondes de santé et les variables d'environnement :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api
  namespace: devops-project
spec:
  replicas: 2
```

```yaml
  selector:
    matchLabels:
      app: flask-api
  template:
    metadata:
      labels:
        app: flask-api
    spec:
      containers:
        - name: flask-api
          image: abderrahmanelns/flask-devops-api:latest
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: flask-config
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
              memory: "128Mi"
              cpu: "200m"
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 30
          readinessProbe:
            httpGet:
```

```
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
```

Points importants du Deployment :

- 2 réplicas : deux instances de l'application tournent simultanément, assurant la haute disponibilité. Si un pod tombe, l'autre continue de servir les requêtes pendant que Kubernetes redémarre le pod défaillant.
- Resource limits : les limites de ressources (memory: 128Mi, cpu: 200m) empêchent un pod mal configuré de consommer toutes les ressources du nœud, protégeant les autres workloads.
- livenessProbe : Kubernetes vérifie l'endpoint /health toutes les 30 secondes. Si le check échoue 3 fois de suite, le pod est redémarré automatiquement.
- readinessProbe : Kubernetes attend que l'endpoint /health retourne 200 avant d'envoyer du trafic vers le pod. Cela évite d'envoyer des requêtes à un pod qui n'est pas encore prêt.
- Variables d'environnement POD_NAME et NODE_NAME : injectées directement depuis les métadonnées Kubernetes (Downward API), permettant à l'application de savoir sur quel pod et quel nœud elle s'exécute.

6.3.4 Service (service.yaml)

Le Service expose l'application à l'extérieur du cluster via un NodePort. Un NodePort ouvre un port spécifique sur chaque nœud du cluster et redirige le trafic vers les pods de l'application :

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
  namespace: devops-project
spec:
  type: NodePort
  selector:
    app: flask-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 30080
```

Le Service de type NodePort a été choisi plutôt qu'un LoadBalancer car ce dernier crée un load balancer AWS qui génère des coûts supplémentaires non couverts par le tier gratuit. Le NodePort permet d'accéder à l'application via le port 30080 sur n'importe quel nœud du cluster.

6.4 Déploiement des Manifestes

Les manifestes sont appliqués dans un ordre précis : le namespace doit exister avant les autres ressources, et le ConfigMap doit être créé avant le Deployment pour que les variables d'environnement soient disponibles au démarrage des pods :

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get pods -n devops-project
NAME                          READY   STATUS    RESTARTS   AGE
flask-api-86794dc585-8b6tv    1/1     Running   0          2m34s
flask-api-86794dc585-k4hg6    1/1     Running   0          2m34s
```

6.5 Démonstration de l'Auto-Guérison

L'une des fonctionnalités les plus importantes de Kubernetes est sa capacité à maintenir automatiquement l'état désiré de l'application. Cette fonctionnalité a été démontrée en supprimant manuellement l'un des pods :

```
# Suppression manuelle d'un pod
kubectl delete pod flask-api-74fdbd5b45-ldklp -n devops-project


# Observation du comportement de Kubernetes
kubectl get pods -n devops-project -w
```

```
^Cuser@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl delete pod -n devops-project $(kubectl get pods -n devops-project -o jsonpath='{.items[0].metadata.name}')
pod "flask-api-74fdbd5b45-ldklp" deleted from devops-project namespace
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get pods -n devops-project -w
NAME                          READY   STATUS    RESTARTS   AGE
flask-api-74fdbd5b45-mrp6r    1/1     Running   0          31m
flask-api-74fdbd5b45-qpgc8    1/1     Running   0          14s
```

Comme on peut le voir, Kubernetes a détecté immédiatement la disparition du pod et en a créé un nouveau automatiquement. Le pod de remplacement (flask-api-74fdbd5b45-qpgc8) est devenu opérationnel en moins de 30 secondes, sans aucune intervention manuelle.

6.6 Vérification de l'Application

Une fois le déploiement effectué, l'application est accessible via l'IP publique du maître sur le port 30080 :

```
# Test de l'endpoint principal
curl http://35.172.240.19:30080/
# Réponse : {"message":"DevOps Project
API","status":"running","version":"1.0.0"}


# Test du health check
curl http://35.172.240.19:30080/health
# Réponse : {"hostname":"flask-api-
...","status":"healthy","timestamp":"..."}


# Test des informations de déploiement
curl http://35.172.240.19:30080/info
# Réponse : {"app":"flask-devops-api","environment":"production",...}
```

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/
{"message":"DevOps Project API","status":"running","version":"1.0.0"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/health
{"hostname":"flask-api-74fdbd5b45-mrp6r","status":"healthy","timestamp":"2026-03-21T21:36:35.855066"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://35.172.240.19:30080/info
{"app":"flask-devops-api","environment":"production","node_name":"ip-10-0-1-124","pod_name":"flask-api-74fdbd5b45-mrp6r"}
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ curl http://3.236.90.164:30080/health
{"hostname":"flask-api-74fdbd5b45-ldklp","status":"healthy","timestamp":"2026-03-21T21:36:53.046451"}
```

7. Pipeline CI/CD avec GitHub Actions

7.1 Concept de CI/CD

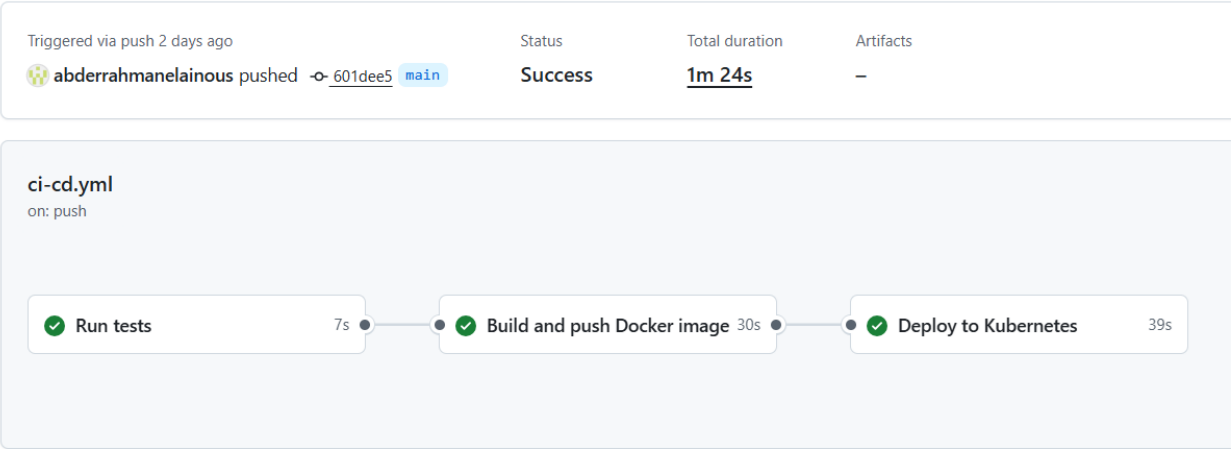
L'Intégration Continue (CI — Continuous Integration) et le Déploiement Continu (CD — Continuous Deployment) sont des pratiques fondamentales du DevOps.

L'Intégration Continue consiste à intégrer fréquemment les modifications de code dans un dépôt partagé, et à vérifier automatiquement que ces modifications ne cassent pas le système existant (via des tests automatisés). Cela permet de détecter les problèmes tôt, quand ils sont encore faciles à corriger.

Le Déploiement Continu va plus loin : non seulement les modifications sont testées automatiquement, mais elles sont également déployées automatiquement en production (ou dans un environnement de staging). Cela permet de livrer de la valeur aux utilisateurs plus rapidement et de manière plus fiable.

7.2 Architecture du Pipeline

Le pipeline CI/CD est déclenché automatiquement à chaque push sur la branche main du dépôt GitHub. Il se compose de trois jobs qui s'exécutent séquentiellement :



Chaque job est conditionné au succès du précédent (grâce à la directive needs). Si les tests échouent, l'image n'est pas construite. Si l'image ne peut pas être publiée, le déploiement n'est pas effectué. Ce mécanisme garantit qu'une version défectueuse ne peut jamais atteindre le cluster Kubernetes.

7.3 Fichier de Configuration du Pipeline (.github/workflows/ci-cd.yml)

Le pipeline est défini dans le fichier .github/workflows/ci-cd.yml :

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  test:
    name: Run tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install flask==3.0.3 pytest
      - name: Run tests
        run: |
          cd app
          python3 -m pytest tests/test_app.py -v
```

```yaml
  build-and-push:
    name: Build and push Docker image
    runs-on: ubuntu-latest
    needs: test
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
```

```
      - name: Build and push image
        uses: docker/build-push-action@v5
        with:
          context: ./app
          platforms: linux/amd64
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/flask-devops-api:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/flask-devops-api:${{
github.sha }}
```

```
  deploy:
    name: Deploy to Kubernetes
    runs-on: ubuntu-latest
    needs: build-and-push
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup SSH key
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.EC2_SSH_KEY }}" > ~/.ssh/devops-project-
key.pem
          chmod 400 ~/.ssh/devops-project-key.pem
          ssh-keyscan -H ${{ secrets.EC2_MASTER_IP }} >>
~/.ssh/known_hosts
      - name: Install kubectl
        run: |
          curl -LO "https://dl.k8s.io/release/$(curl -L -s ...) "
          sudo install -o root -g root -m 0755 kubectl
/usr/local/bin/kubectl
      - name: Get kubeconfig from master
        run: |
          mkdir -p ~/.kube
          scp -i ~/.ssh/devops-project-key.pem \
            ubuntu@${{ secrets.EC2_MASTER_IP
}}:/etc/rancher/k3s/k3s.yaml \
            ~/.kube/config
          sed -i 's/127.0.0.1/${{ secrets.EC2_MASTER_IP }}/g'
~/.kube/config
      - name: Deploy to Kubernetes
        run: |
```

```
          kubectl apply -f k8s/namespace.yaml
          kubectl apply -f k8s/configmap.yaml
          kubectl apply -f k8s/deployment.yaml
          kubectl apply -f k8s/service.yaml
          kubectl rollout restart deployment/flask-api -n devops-
project
          kubectl rollout status deployment/flask-api -n devops-project
--timeout=120s
```

7.4 Gestion des Secrets

La gestion des secrets est un aspect critique de la sécurité dans les pipelines CI/CD. GitHub Actions fournit un mécanisme de stockage chiffré pour les informations sensibles (secrets), qui sont injectées comme variables d'environnement lors de l'exécution du pipeline :

Nom du Secret	Contenu	Utilisation dans le pipeline
DOCKERHUB_USERNAME	Nom d'utilisateur Docker Hub	Authentification pour push de l'image
DOCKERHUB_TOKEN	Token d'accès Docker Hub (Read/Write)	Authentification sécurisée (pas le mot de passe)
EC2_MASTER_IP	IP publique du nœud maître (35.172.240.19)	Connexion SSH et configuration kubectl
EC2_SSH_KEY	Contenu de la clé privée RSA (.pem)	Authentification SSH pour le déploiement

Bonne pratique sécurité

Les secrets GitHub sont chiffrés au repos et en transit. Ils ne sont jamais affichés dans les logs de GitHub Actions (remplacés par ***). Un token Docker Hub a été utilisé à la place du mot de passe principal, permettant de révoquer l'accès du pipeline sans changer le mot de passe du compte.

7.5 Déclenchement Manuel

En plus du déclenchement automatique sur push, le pipeline peut être déclenché manuellement via la directive workflow_dispatch. Cette fonctionnalité est utile pour forcer un redéploiement sans modifier le code, par exemple après un redémarrage des instances EC2.

7.6 Résultat du Pipeline

Le pipeline a été exécuté avec succès, confirmant le bon fonctionnement de l'ensemble de la chaîne CI/CD :

Job	Durée	Statut	Actions effectuées
Run tests	~10s	Succès	3 tests pytest passés
Build and push Docker image	~32s	Succès	Image construite et publiée sur Docker Hub
Deploy to Kubernetes	~37s	Succès	Manifestes appliqués, rollout terminé

8. Validation et Tests

8.1 Stratégie de Validation

Une validation complète de l'infrastructure a été effectuée à plusieurs niveaux, de la couche infrastructure jusqu'à l'application :

Niveau	Outil de validation	Ce qui est vérifié
Infrastructure	terraform plan / terraform show	Ressources AWS créées correctement
Connectivité	ssh, ansible ping	Accès SSH aux instances
Configuration	Ansible PLAY RECAP	Tous les logiciels installés
Cluster K3s	kubectl get nodes	Tous les nœuds en état Ready
Application	curl	Endpoints HTTP répondent correctement
Tests unitaires	pytest	Logique applicative validée
Pipeline CI/CD	GitHub Actions	Pipeline complet vert
Auto-guérison	kubectl delete pod	Kubernetes recrée le pod automatiquement

8.2 Validation de l'Infrastructure Terraform

```
# Lister toutes les ressources gérées par Terraform
terraform state list
# Résultat attendu :
# aws_instance.master
# aws_instance.worker
# aws_internet_gateway.main
# aws_route_table.public
# aws_route_table_association.public
# aws_security_group.k3s
# aws_subnet.public
# aws_vpc.main

# Afficher les outputs
terraform output
# master_public_ip = "35.172.240.19"
# worker_public_ip = "3.236.90.164"
# ssh_master = "ssh -i ~/.ssh/devops-project-key.pem
ubuntu@35.172.240.19"
```

8.3 Validation du Cluster Kubernetes

```
# État des nœuds
kubectl get nodes -o wide
# NAME              STATUS   ROLES           AGE   VERSION
# ip-10-0-1-178     Ready    control-plane   2h    v1.34.5+k3s1
# ip-10-0-1-124     Ready    <none>          2h    v1.34.5+k3s1

# État des pods
kubectl get pods -n devops-project
# NAME                          READY   STATUS    RESTARTS   AGE
# flask-api-74fdbd5b45-mrp6r    1/1     Running   0          31m
# flask-api-74fdbd5b45-qpgc8    1/1     Running   0          14s

# État du service
kubectl get service -n devops-project
# NAME                TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)
# flask-api-service   NodePort   10.43.145.198   <none>
80:30080/TCP
```

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE   VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
ip-10-0-1-124  Ready    <none>          18h   v1.34.5+k3s1   10.0.1.124    <none>        Ubuntu 22.04.5 LTS   6.8.0-1050-aws   containerd://2.1.5-k3s1
ip-10-0-1-178  Ready    control-plane   19h   v1.34.5+k3s1   10.0.1.178    <none>        Ubuntu 22.04.5 LTS   6.8.0-1050-aws   containerd://2.1.5-k3s1
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get pods --all-namespaces
NAMESPACE        NAME                                      READY   STATUS    RESTARTS   AGE
devops-project   flask-api-74fdbd5b45-ldklp                1/1     Running   0          4m9s
devops-project   flask-api-74fdbd5b45-mrp6r                1/1     Running   0          4m23s
kube-system      coredns-695cbbfcb9-2t5nj                  1/1     Running   0          19h
kube-system      local-path-provisioner-546dfc6456-hff71   1/1     Running   0          19h
kube-system      metrics-server-c8774f4f4-fx6bx            1/1     Running   0          19h
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get pods -n devops-project -o wide
NAME                          READY   STATUS    RESTARTS   AGE     IP          NODE            NOMINATED NODE   READINESS GATES
flask-api-74fdbd5b45-ldklp    1/1     Running   0          4m19s   10.42.0.7   ip-10-0-1-178   <none>           <none>
flask-api-74fdbd5b45-mrp6r    1/1     Running   0          4m33s   10.42.1.4   ip-10-0-1-124   <none>           <none>
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get service -n devops-project
NAME                TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
flask-api-service   NodePort   10.43.145.198   <none>        80:30080/TCP   18h
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get deployment -n devops-project
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-api   2/2     2            2           18h
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl get configmap -n devops-project
NAME               DATA   AGE
flask-config       1      18h
kube-root-ca.crt   1      18h
```

8.4 Validation de l'Application

```
# Test complet de tous les endpoints
echo '=== ENDPOINT PRINCIPAL ===' && curl -s
http://35.172.240.19:30080/
echo '=== HEALTH CHECK ===' && curl -s
http://35.172.240.19:30080/health
echo '=== INFO POD ===' && curl -s http://35.172.240.19:30080/info
```

8.5 Validation de la Mémoire Swap

```
# Vérification du swap sur le maître
ssh -i ~/.ssh/devops-project-key.pem ubuntu@35.172.240.19 'free -h'
#               total    used    free  shared  buff/cache  available
# Mem:          914Mi   350Mi   100Mi    2Mi       463Mi       460Mi
# Swap:         2.0Gi     0B    2.0Gi
```

8.6 Validation des Logs de l'Application

Les logs de l'application peuvent être consultés directement depuis Kubernetes, permettant de vérifier le bon fonctionnement de Gunicorn et de l'application Flask :

```
user@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl logs -n devops-project -l app=flask-api -f
[2026-03-21 21:31:08 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-03-21 21:31:08 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-03-21 21:31:08 +0000] [1] [INFO] Using worker: sync
[2026-03-21 21:31:08 +0000] [7] [INFO] Booting worker with pid: 7
[2026-03-21 21:31:09 +0000] [8] [INFO] Booting worker with pid: 8
[2026-03-21 21:31:24 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-03-21 21:31:24 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-03-21 21:31:24 +0000] [1] [INFO] Using worker: sync
[2026-03-21 21:31:24 +0000] [7] [INFO] Booting worker with pid: 7
[2026-03-21 21:31:24 +0000] [8] [INFO] Booting worker with pid: 8
^Cuser@DESKTOP-HMJ54M8:/mnt/c/Users/user/devops-project/ansible$ kubectl logs -n devops-project -l app=flask-api -f
[2026-03-21 21:31:24 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-03-21 21:31:24 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-03-21 21:31:24 +0000] [1] [INFO] Using worker: sync
[2026-03-21 21:31:24 +0000] [7] [INFO] Booting worker with pid: 7
[2026-03-21 21:31:24 +0000] [8] [INFO] Booting worker with pid: 8
[2026-03-21 21:31:08 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-03-21 21:31:08 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-03-21 21:31:08 +0000] [1] [INFO] Using worker: sync
[2026-03-21 21:31:08 +0000] [7] [INFO] Booting worker with pid: 7
[2026-03-21 21:31:09 +0000] [8] [INFO] Booting worker with pid: 8
```

9. Difficultés Rencontrées et Solutions

Ce chapitre documente les principaux problèmes techniques rencontrés lors de la réalisation du projet, ainsi que les solutions qui ont permis de les résoudre. Cette section est particulièrement utile pour quiconque souhaite reproduire l'infrastructure et éviter les mêmes écueils.

9.1 Instance t2.micro Non Éligible au Tier Gratuit

9.1.1 Problème

Lors de la première exécution de terraform apply, les instances EC2 ne pouvaient pas être créées avec le type t2.micro :

```
Error: creating EC2 Instance: InvalidParameterCombination:
The specified instance type is not eligible for Free Tier.
For a list of Free Tier instance types, run 'describe-instance-types'
with the filter 'free-tier-eligible=true'.
```

9.1.2 Cause

AWS a changé son offre de tier gratuit pour les nouveaux comptes créés après 2023. Le type d'instance t2.micro n'est plus éligible dans certaines régions ; c'est désormais t3.micro qui bénéficie du tier gratuit.

9.1.3 Solution

Vérification du type d'instance éligible avec la commande AWS CLI, puis mise à jour de la variable instance_type dans variables.tf :

```
aws ec2 describe-instance-types \
  --filters "Name=free-tier-eligible,Values=true" \
  --query "InstanceTypes[*].InstanceType" \
  --output table
# Résultat : t3.micro, t3.small, t4g.micro, etc.
```

9.2 Erreur de Certificat TLS K3s

9.2.1 Problème

Après l'installation initiale de K3s, la commande kubectl get nodes depuis WSL2 échouait avec une erreur de certificat TLS :

```
E0321 tls: failed to verify certificate:
x509: certificate is valid for 10.0.1.178, 10.43.0.1, 127.0.0.1, ::1,
not 35.172.240.19
```

9.2.2 Cause

Par défaut, K3s génère son certificat TLS avec uniquement l'IP privée du serveur et l'IP loopback (127.0.0.1). L'IP publique n'est pas incluse dans les Subject Alternative Names (SANs) du certificat, ce qui cause le rejet par les clients qui tentent de se connecter via l'IP publique.

9.2.3 Solution

Réinstallation de K3s en passant l'option --tls-san avec l'IP publique du serveur. Cette option ajoute l'IP publique aux SANs du certificat TLS généré :

```
sudo /usr/local/bin/k3s-uninstall.sh


curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server \
  --disable traefik \
  --write-kubeconfig-mode 644 \
  --tls-san 35.172.240.19 \
  --tls-san 10.0.1.178" sh -
```

9.3 Problème d'Image Multi-Architecture Docker

9.3.1 Problème

Après avoir pushé l'image Docker sur Docker Hub et tenté de la déployer sur Kubernetes, les pods restaient en état ImagePullBackOff :

```
failed to pull and unpack image
"docker.io/abderrahmanelns/flask-devops-api:latest":
failed to resolve reference: not found
```

9.3.2 Cause

Docker Buildx, l'outil de construction multi-plateforme, crée par défaut une image manifest list (liste de manifestes) qui référence plusieurs images pour différentes architectures (amd64, arm64, etc.). Containerd, le runtime de conteneurs utilisé par K3s, ne peut pas résoudre cette liste de manifestes dans certaines configurations.

9.3.3 Solution

Reconstruction de l'image en forçant une seule architecture avec l'option --platform linux/amd64 :

```
docker build --platform linux/amd64 \
  -t abderrahmanelns/flask-devops-api:latest .


docker push abderrahmanelns/flask-devops-api:latest
```

9.4 Clé SSH Corrompue dans les Secrets GitHub

9.4.1 Problème

Le job de déploiement du pipeline CI/CD échouait systématiquement avec l'erreur suivante :

```
Load key "/home/runner/.ssh/devops-project-key.pem": error in libcrypto
ubuntu@***: Permission denied (publickey).
scp: Connection closed
Error: Process completed with exit code 255.
```

9.4.2 Cause

Lors du copier-coller de la clé privée RSA dans l'interface GitHub (champ texte du secret EC2_SSH_KEY), les fins de lignes ont été corrompues. Le navigateur ou l'interface GitHub a introduit des caractères supplémentaires ou modifié le format de la clé, la rendant invalide pour libcrypto/OpenSSL.

9.4.3 Solution

Utilisation de GitHub CLI (gh) pour définir le secret directement depuis le fichier, sans passer par le copier-coller. Cette méthode préserve les fins de lignes Unix (LF) et l'intégrité complète de la clé :

```
# Installation de GitHub CLI dans WSL2
sudo apt install gh -y

# Authentification
gh auth login

# Définition du secret directement depuis le fichier
gh secret set EC2_SSH_KEY \
  --repo abderrahmanelainous/devops-project \
  < ~/.ssh/devops-project-key.pem
```

9.5 Problème de Connectivité Ansible

9.5.1 Problème

Lors des premières exécutions du playbook Ansible, le résultat affichait changed=0 alors que les logiciels n'étaient pas installés sur les machines.

9.5.2 Cause

Les fichiers main.yml des rôles Ansible avaient été créés comme fichiers vides (via la commande touch). Ansible reconnaissait les rôles mais n'avait aucune tâche à exécuter.

9.5.3 Solution

Recréation des répertoires tasks manquants et écriture du contenu complet dans chaque fichier main.yml :

```
mkdir -p ansible/roles/common/tasks
mkdir -p ansible/roles/docker/tasks
mkdir -p ansible/roles/k3s/tasks

# Vérification du contenu des fichiers
wc -l ansible/roles/*/tasks/main.yml
# 71 ansible/roles/common/tasks/main.yml
# 52 ansible/roles/docker/tasks/main.yml
# 49 ansible/roles/k3s/tasks/main.yml
```

9.6 Tableau Récapitulatif des Problèmes

Problème	Impact	Solution
t2.micro non éligible tier gratuit	Bloquant — instances non créées	Utiliser t3.micro
Certificat TLS K3s IP publique	Bloquant — kubectl refusé	Option --tls-san au démarrage K3s
Image Docker multi-architecture	Bloquant — pods ImagePullBackOff	Option --platform linux/amd64
Clé SSH corrompue dans GitHub	Bloquant — déploiement CI/CD échoue	GitHub CLI pour set secret depuis fichier
Rôles Ansible vides	Bloquant — aucune installation effectuée	Recréer les fichiers main.yml avec contenu
Login Docker Hub interactif sur EC2	Bloquant — interface navigateur sur serveur	docker login -u username (mode CLI)

10. Conclusion et Perspectives

10.1 Bilan du Projet

Ce projet a permis de réaliser avec succès une infrastructure DevOps complète et fonctionnelle, en respectant toutes les contraintes du cahier des charges. L'ensemble des objectifs principaux a été atteint :

- Infrastructure as Code avec Terraform : la totalité de l'infrastructure AWS (VPC, sous-réseau, Internet Gateway, groupe de sécurité, 2 instances EC2) est définie dans des fichiers de configuration Terraform versionnés dans Git, permettant une reproductibilité et une traçabilité complètes.
- Automatisation de la configuration avec Ansible : Docker, K3s et la mémoire swap sont installés et configurés automatiquement sur les deux instances EC2 via des playbooks Ansible structurés en rôles réutilisables.
- Cluster Kubernetes opérationnel : un cluster K3s avec un nœud maître et un nœud worker est déployé et opérationnel, avec la fonctionnalité d'auto-guérison démontrée (suppression et recréation automatique d'un pod en moins de 30 secondes).
- Application conteneurisée : une API REST Flask est conteneurisée avec Docker selon les bonnes pratiques (image slim, utilisateur non-root, healthcheck, Gunicorn) et déployée avec 2 réplicas sur Kubernetes.
- Pipeline CI/CD entièrement automatisé : le pipeline GitHub Actions enchaîne tests unitaires, construction de l'image Docker, publication sur Docker Hub et déploiement sur Kubernetes en moins de 80 secondes après chaque commit.
- Documentation complète : ce rapport documente en détail l'ensemble du projet, y compris les difficultés rencontrées et les solutions adoptées.

10.2 Compétences Acquises

La réalisation de ce projet a permis d'acquérir et d'approfondir un large éventail de compétences DevOps :

- Terraform : provisionnement d'infrastructure cloud, gestion de l'état, variables et outputs.
- Ansible : organisation en rôles, inventaires dynamiques, gestion de l'idempotence, modules apt, file, command et lineinfile.
- Docker : écriture de Dockerfiles optimisés, gestion des layers, publication sur Docker Hub, construction multi-plateforme.
- Kubernetes : déploiements, services NodePort, ConfigMaps, probes de santé, gestion des ressources et auto-guérison.
- GitHub Actions : workflows YAML, gestion des secrets, jobs séquentiels et parallèles, actions réutilisables.
- Sécurité : gestion des clés SSH, tokens d'accès, IAM AWS, groupes de sécurité et utilisateurs non-root dans les conteneurs.
- Résolution de problèmes : diagnostic d'erreurs TLS, de problèmes d'architecture d'images Docker, de connectivité SSH et de configuration Ansible.

10.3 Perspectives et Améliorations Futures

Plusieurs améliorations pourraient être apportées pour rendre cette infrastructure encore plus robuste et complète :

10.3.1 Monitoring avec Prometheus et Grafana

L'implémentation d'une stack de monitoring complète est la priorité numéro un pour une version future. Prometheus collecterait les métriques de l'application et de l'infrastructure (utilisation CPU, mémoire, requêtes HTTP, latence), tandis que Grafana fournirait des tableaux de bord visuels. Sur des instances avec plus de RAM (t3.small avec 2 Go ou t3.medium avec 4 Go), cette fonctionnalité serait facilement implémentable via Helm charts.

10.3.2 Contrôleur Ingress

Remplacer le Service NodePort par un contrôleur Ingress (comme Nginx Ingress ou Traefik) permettrait de gérer plusieurs applications sur le même cluster, d'utiliser des noms de domaine, et d'implémenter HTTPS avec Let's Encrypt (via cert-manager).

10.3.3 Charts Helm

Packager les manifestes Kubernetes dans des charts Helm permettrait de gérer plus facilement les versions de l'application, les environnements (staging/production) et les configurations personnalisées.

10.3.4 Multi-Environnements

La mise en place d'environnements séparés (développement, staging, production) avec des branches Git dédiées et des déploiements automatiques vers chaque environnement rendrait le pipeline plus professionnel et sécurisé.

10.3.5 Gestion des Secrets avec Vault

L'utilisation de HashiCorp Vault pour la gestion centralisée des secrets (clés API, mots de passe de bases de données, certificats) remplacerait les secrets GitHub et les variables d'environnement, offrant une traçabilité, un contrôle d'accès fin et une rotation automatique des secrets.

10.3.6 Base de Données

L'ajout d'une base de données persistante (PostgreSQL via un StatefulSet Kubernetes ou Amazon RDS) transformerait l'API stateless actuelle en une vraie application backend avec persistence des données.

10.4 Mot de Fin

Ce projet a démontré qu'il est possible de mettre en place une infrastructure DevOps professionnelle complète en utilisant exclusivement des outils open source et des services cloud gratuits. Les principes d'Infrastructure as Code, d'automatisation, de conteneurisation et de déploiement continu qui ont été mis en pratique ici sont exactement ceux utilisés par les équipes DevOps dans les grandes entreprises technologiques.

La valeur principale de ce projet ne réside pas dans la complexité de l'application déployée, mais dans la robustesse et la reproductibilité de l'infrastructure qui la supporte. N'importe qui peut désormais cloner le dépôt GitHub, exécuter quatre commandes (terraform apply, ansible-playbook, kubectl apply, et un push sur GitHub), et obtenir une infrastructure identique fonctionnelle en moins de 30 minutes.

11. Annexes

Annexe A — Structure Complète du Projet

```
devops-project/
├── .github/
│   └── workflows/
│       └── ci-cd.yml          # Pipeline GitHub Actions
├── app/
│   ├── app.py                 # Code source Flask
│   ├── Dockerfile             # Définition de l'image Docker
│   ├── requirements.txt       # Dépendances Python
│   └── tests/
│       └── test_app.py        # Tests unitaires pytest
├── terraform/
│   ├── main.tf                # Ressources AWS principales
│   ├── variables.tf           # Variables Terraform
│   └── outputs.tf             # Sorties (IPs, commandes SSH)
├── ansible/
│   ├── inventory.ini          # Inventaire des machines
│   ├── playbook.yml           # Playbook principal
│   └── roles/
│       ├── common/tasks/main.yml   # Système + swap
│       ├── docker/tasks/main.yml   # Docker CE
│       └── k3s/tasks/main.yml      # K3s cluster
├── k8s/
│   ├── namespace.yaml         # Namespace Kubernetes
│   ├── configmap.yaml         # Variables de configuration
│   ├── deployment.yaml        # Déploiement Flask (2 réplicas)
│   └── service.yaml           # Service NodePort :30080
├── docs/
│   └── setup-guide.md         # Guide de reproduction
├── k3s.yaml                   # Kubeconfig (git-ignored en prod)
├── .gitignore
└── README.md
```

Annexe B — Commandes de Référence Complètes

B.1 Terraform

```
# Initialiser Terraform
```

```
cd terraform && terraform init

# Valider la configuration
terraform validate

# Afficher le plan d'exécution
terraform plan

# Appliquer les changements
terraform apply

# Lister les ressources gérées
terraform state list

# Afficher les outputs
terraform output

# Détruire toute l'infrastructure
terraform destroy
```

B.2 Ansible

```
# Tester la connectivité
ansible all -i inventory.ini -m ping

# Vérifier la syntaxe du playbook
ansible-playbook -i inventory.ini playbook.yml --syntax-check

# Exécuter le playbook (mode verbose)
ansible-playbook -i inventory.ini playbook.yml -v

# Exécuter un seul rôle (avec tags)
ansible-playbook -i inventory.ini playbook.yml --tags k3s
```

B.3 Kubectl

```
# Configurer kubectl
export KUBECONFIG=/path/to/k3s.yaml

# État du cluster
kubectl get nodes -o wide
```

```
kubectl cluster-info

# Gestion des pods
kubectl get pods -n devops-project
kubectl get pods -n devops-project -o wide
kubectl describe pod POD_NAME -n devops-project
kubectl logs -n devops-project -l app=flask-api
kubectl logs -n devops-project -l app=flask-api -f

# Scaling
kubectl scale deployment flask-api -n devops-project --replicas=3

# Restart déploiement
kubectl rollout restart deployment/flask-api -n devops-project
kubectl rollout status deployment/flask-api -n devops-project

# Suppression
kubectl delete namespace devops-project
```

B.4 Gestion des Instances EC2

```
# Arrêter les instances (économiser le quota tier gratuit)
aws ec2 stop-instances --instance-ids \
  $(aws ec2 describe-instances \
    --filters 'Name=tag:Project,Values=devops-project' \
    --query 'Reservations[].Instances[].InstanceId' \
    --output text)

# Redémarrer les instances
aws ec2 start-instances --instance-ids \
  $(aws ec2 describe-instances \
    --filters 'Name=tag:Project,Values=devops-project' \
    --query 'Reservations[].Instances[].InstanceId' \
    --output text)

# Récupérer les nouvelles IPs après redémarrage
aws ec2 describe-instances \
  --filters 'Name=tag:Project,Values=devops-project' \
  --query
'Reservations[].Instances[].[Tags[?Key==Name].Value|[0],PublicIpAddress
]' \
```

```
    --output table
```

Annexe C — Guide de Reproduction

Ce guide permet de reproduire l'infrastructure complète depuis zéro.

C.1 Prérequis Locaux

- Windows avec WSL2 (Ubuntu 22.04) installé
- Git Bash installé
- AWS CLI v2 installé et configuré (aws configure)
- Terraform 1.7.x installé
- kubectl installé dans WSL2
- Ansible installé dans WSL2 (sudo apt install ansible)
- Compte AWS avec accès tier gratuit
- Compte GitHub
- Compte Docker Hub

C.2 Étapes de Reproduction

25. Cloner le dépôt : git clone https://github.com/abderrahmanelainous/devops-project.git
26. Créer la paire de clés SSH AWS : aws ec2 create-key-pair --key-name devops-project-key --query 'KeyMaterial' --output text > ~/.ssh/devops-project-key.pem && chmod 400 ~/.ssh/devops-project-key.pem
27. Récupérer l'AMI Ubuntu 22.04 : aws ec2 describe-images --owners 099720109477 --filters 'Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*' --query 'sort_by(Images, &CreationDate)[-1].ImageId' --output text
28. Mettre à jour variables.tf avec l'AMI ID récupéré
29. Vérifier le type d'instance eligible : aws ec2 describe-instance-types --filters 'Name=free-tier-eligible,Values=true'
30. Provisionner l'infrastructure : cd terraform && terraform init && terraform apply
31. Mettre à jour ansible/inventory.ini avec les IPs des instances créées
32. Copier la clé SSH dans WSL2 : cp /mnt/c/Users/user/.ssh/devops-project-key.pem ~/.ssh/ && chmod 400 ~/.ssh/devops-project-key.pem
33. Exécuter le playbook Ansible : cd ansible && ansible-playbook -i inventory.ini playbook.yml
34. Récupérer et configurer kubectl : scp -i ~/.ssh/devops-project-key.pem ubuntu@MASTER_IP:/etc/rancher/k3s/k3s.yaml ~/devops-project/k3s.yaml && sed -i 's/127.0.0.1/MASTER_IP/g' ~/devops-project/k3s.yaml
35. Déployer l'application : kubectl apply -f k8s/
36. Configurer les secrets GitHub (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, EC2_MASTER_IP, EC2_SSH_KEY via gh secret set)
37. Pousser le code pour déclencher le pipeline : git push origin main

38. Vérifier le déploiement : kubectl get pods -n devops-project && curl http://MASTER_IP:30080/health

Annexe D — Ressources et Références

- Documentation Terraform AWS Provider : https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- Documentation Ansible : https://docs.ansible.com/
- Documentation K3s : https://docs.k3s.io/
- Documentation GitHub Actions : https://docs.github.com/en/actions
- Documentation Kubernetes : https://kubernetes.io/docs/
- Documentation Docker : https://docs.docker.com/
- AWS Free Tier : https://aws.amazon.com/free/